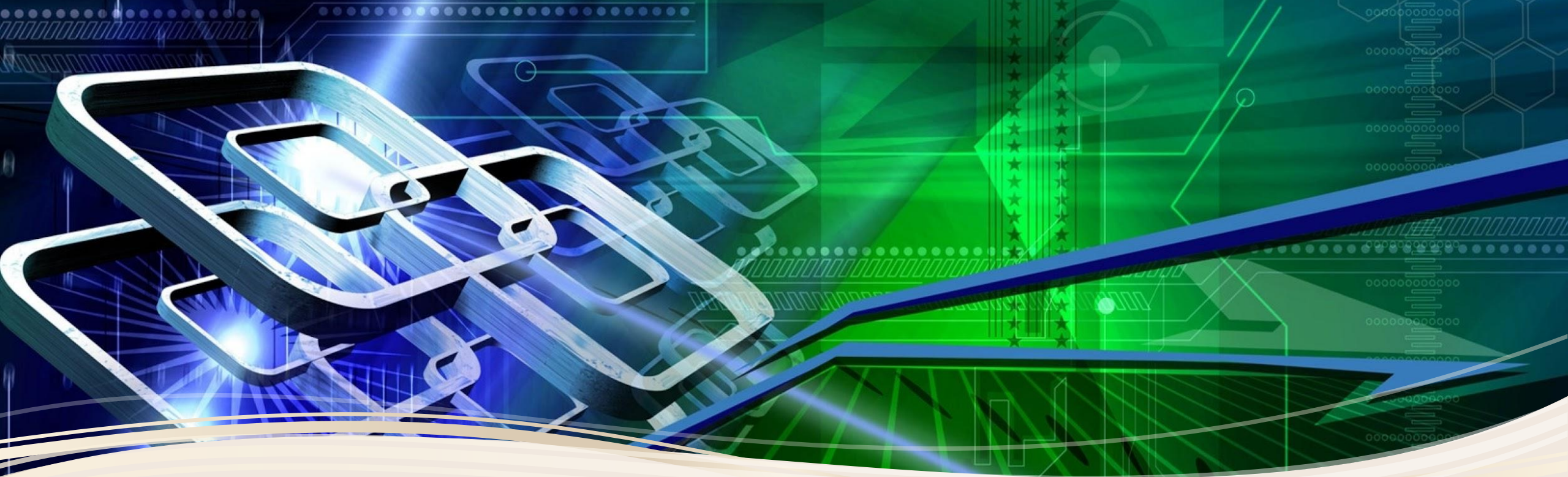




IN242 - Kecerdasan Mesin

Chapter 8

Dimensionality Reduction

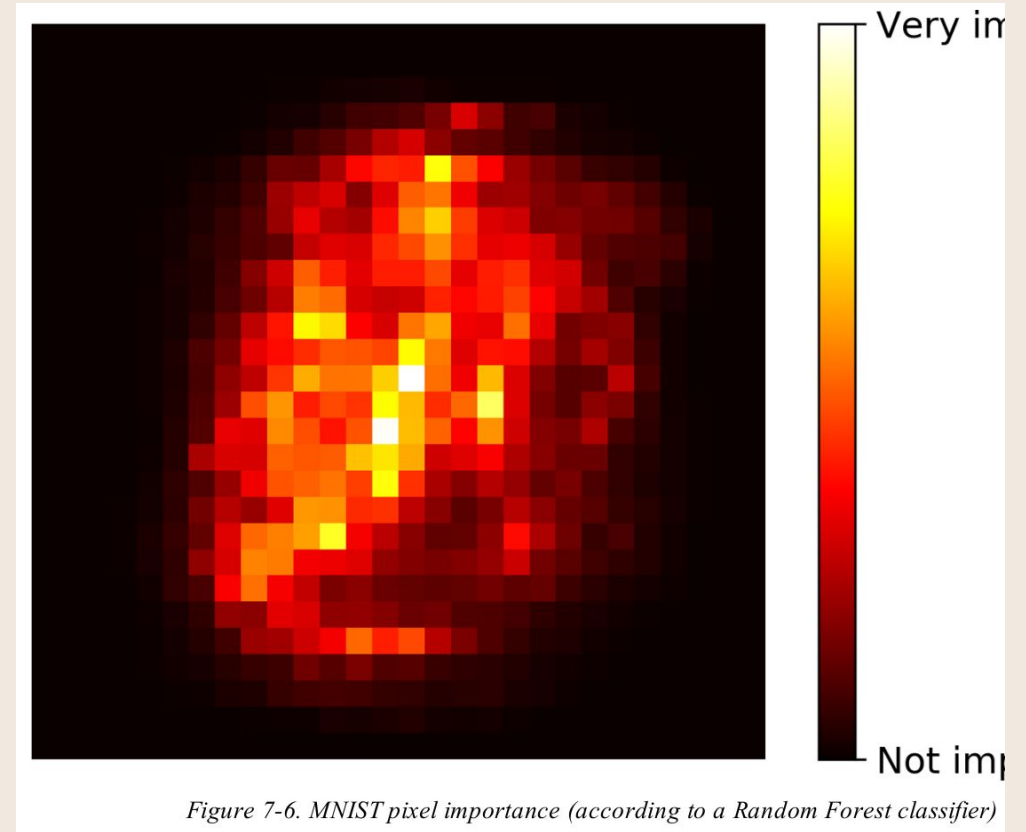
Chapter 8: Dimensionality Reduction, The Curse of Dimensionality, Main Approaches for Dimensionality Reduction, Projection, Manifold Learning

Motivation (1/2)

- Many Machine Learning problems involve thousands or even millions of features for each training instance.
- Not only do all these features make training extremely slow, but they can also make it much harder to find a good solution, as we will see.
- This problem is often referred to as the **curse of dimensionality**.

Motivation (2/2)

- Fortunately, in real-world problems, it is often possible to reduce the number of features considerably, turning an intractable problem into a tractable one.
- For example, consider the MNIST images (introduced in Chapter 3): the pixels on the image borders are almost always white, so you could completely drop these pixels from the training set without losing much information.
- Figure 7-6 confirms that these pixels are utterly unimportant for the classification task



Dimensionality Reduction

Dimensionality reduction is the transformation of data from a high dimensional space into a low dimensional space so that low dimensional representation retains some meaningful properties of the original data.

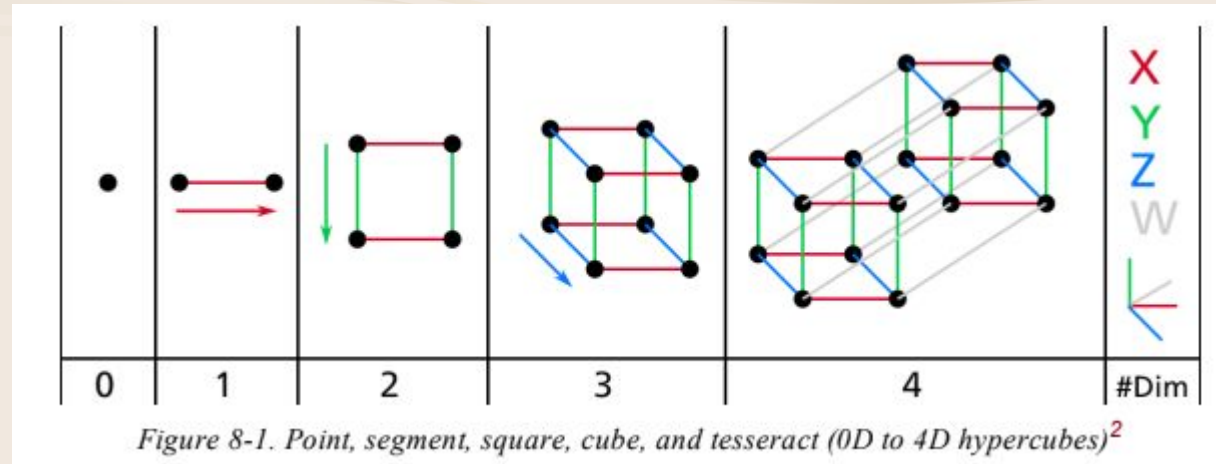
Dimensionality reduction is extremely useful for data visualization and speeding up training.

The Curse of Dimensionality

it refers to a set of problems that arise when working with high-dimensional data.

Some of the difficulties that come with high dimensional data manifest during analyzing or visualizing the data to identify patterns, and some manifest while training machine learning models.

Example of high dimensional data.



A basic 4D hypercube is incredibly hard to picture in our minds. One solution to the curse of dimensionality could be to increase the size of the training set to reach a sufficient density of training instances.

Unfortunately, in practice, the number of training instances required to reach a given density grows exponentially with the number of dimensions.

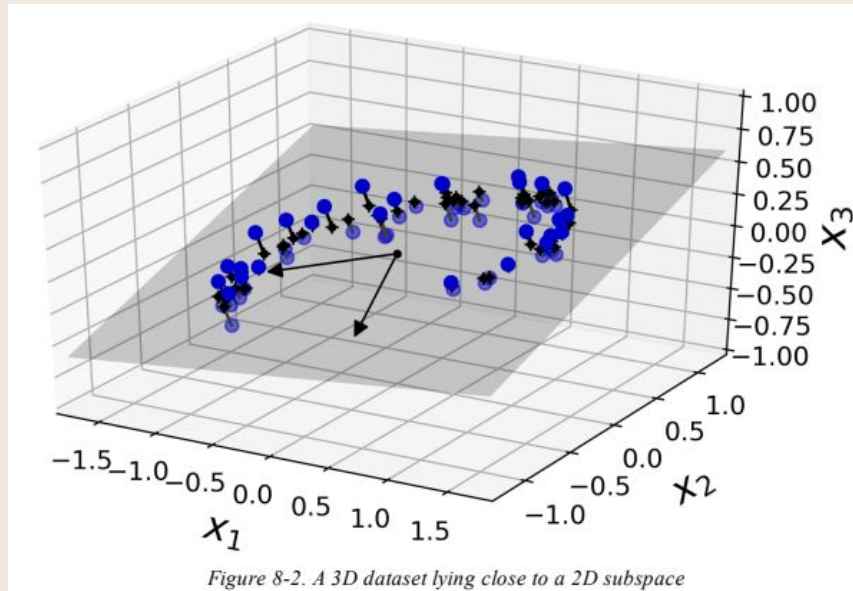
Main Approaches for Dimensionality Reduction

Two main approaches to reducing dimensionality:

Projection and **Manifold Learning**.

Projection

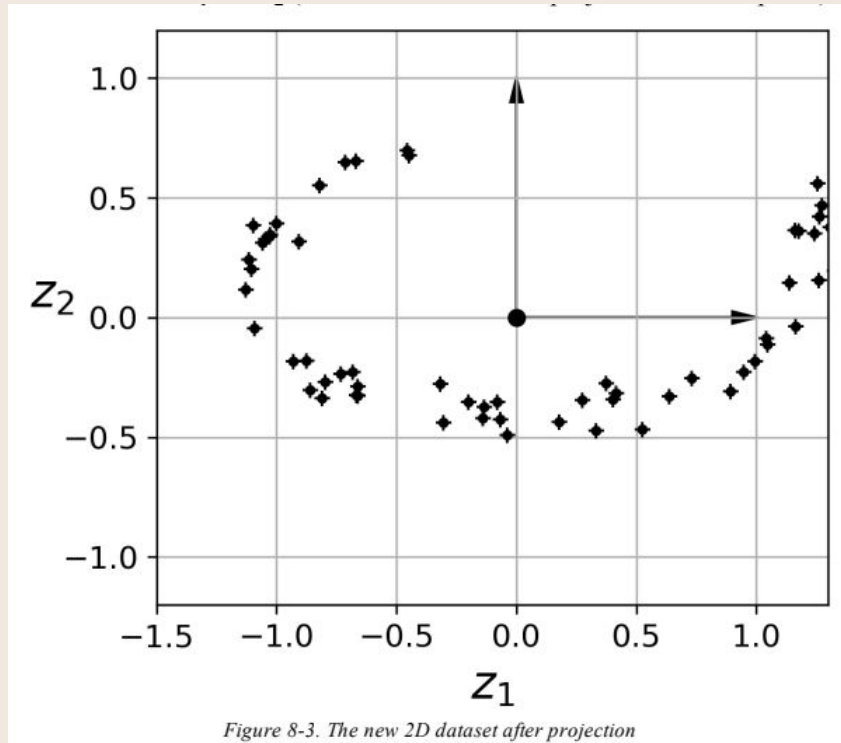
An example of a 3D dataset represented by circles.



This is a lower-dimensional (2D) subspace of the high-dimensional (3D) space.

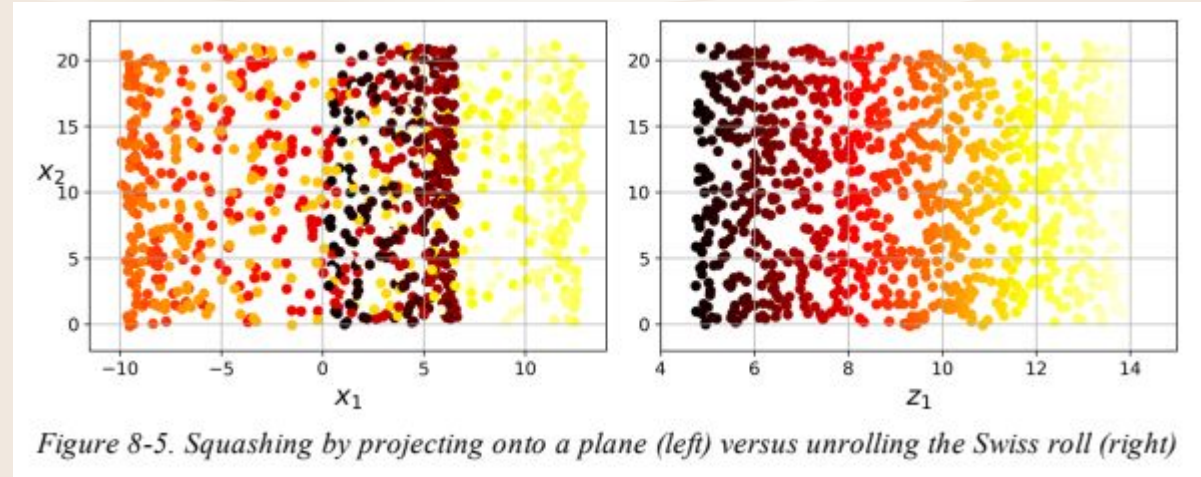
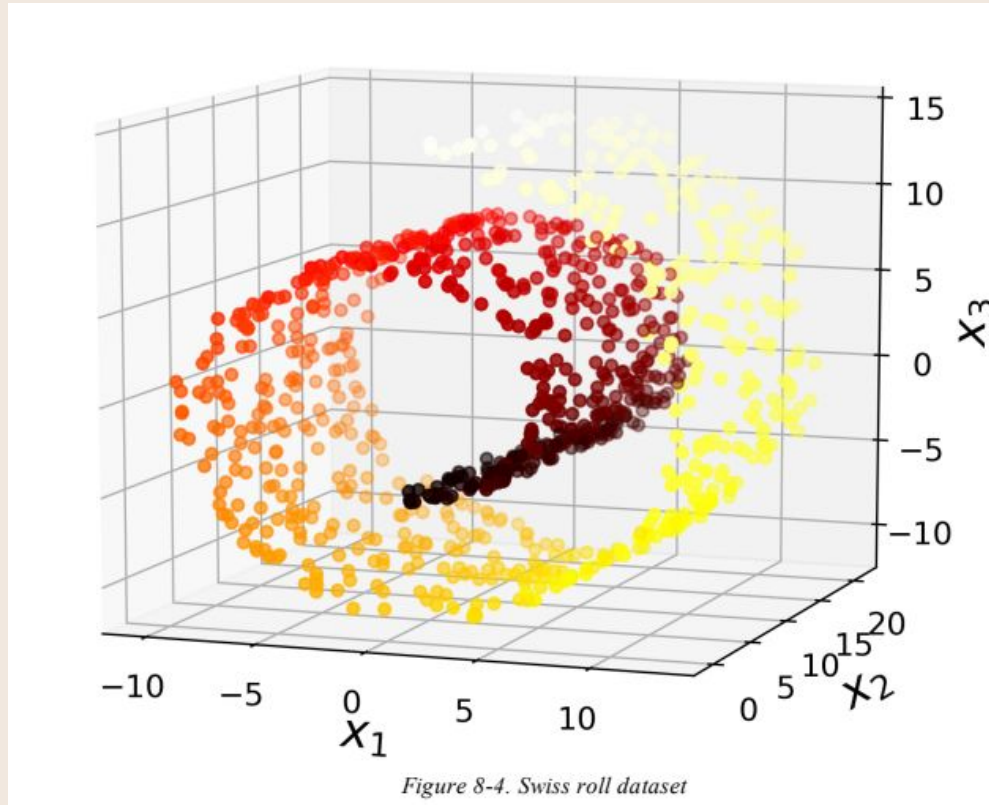
Project every training instance perpendicularly onto this subspace (as represented by the short lines connecting the instances to the plane).

Reduced the dataset's dimensionality from 3D to 2D.



Note that the axes correspond to new features z_1 and z_2 (the coordinates of the projections on the plane).

The subspace may twist and turn, such as in the famous Swiss roll toy dataset



Simply projecting onto a plane (e.g., by dropping x) would squash different layers of the Swiss roll together and unroll the Swiss roll to obtain the 2D dataset on the right side.

Manifold Learning

Dimensionality reduction algorithms work by modeling the manifold on which the training instances lie, this is called Manifold Learning. It relies on the manifold assumption, also called the manifold hypothesis, which holds that most real-world high-dimensional datasets lie close to a much lower-dimensional manifold.

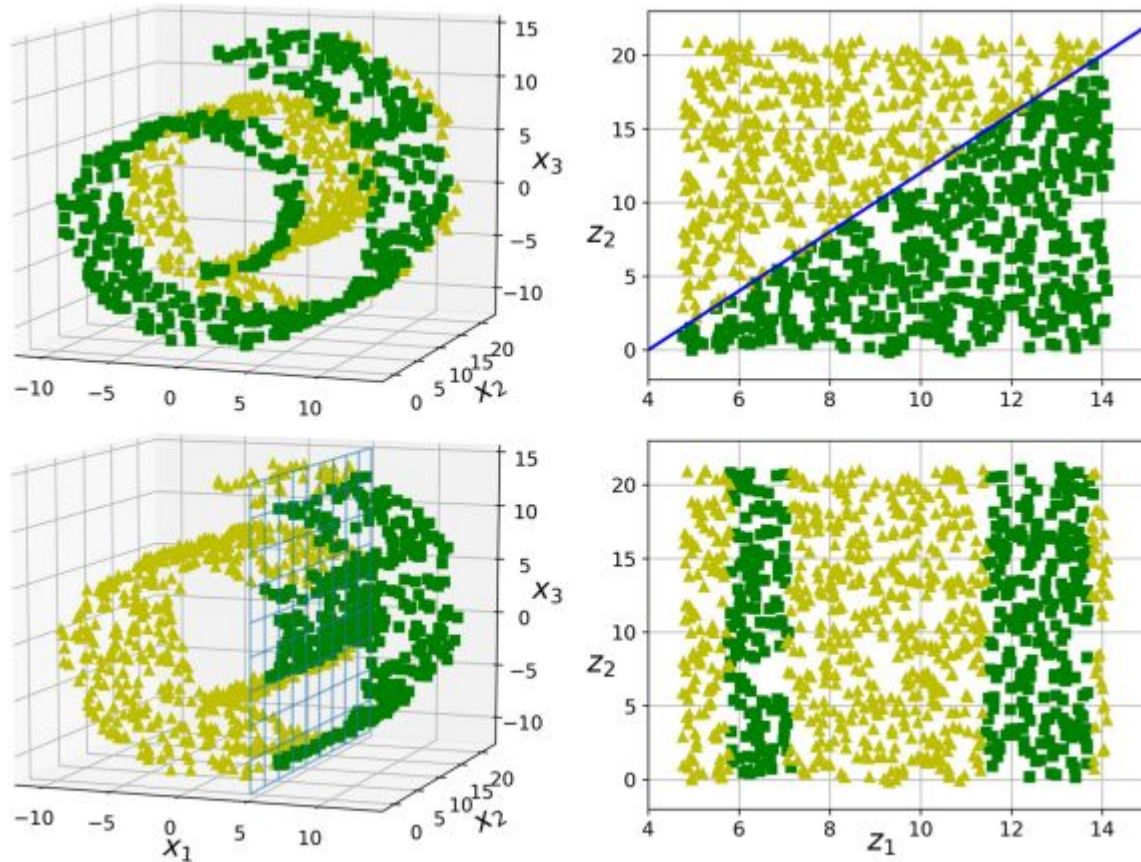


Figure 8-6. The decision boundary may not always be simpler with lower dimensions

The task at hand will be simpler if expressed in the lower-dimensional space of the manifold.

For example, in the top row of Figure 8-6, the Swiss roll is split into two classes: in the 3D space (on the left), the decision boundary would be complex, but in the 2D unrolled manifold space (on the right), the decision boundary is a straight line.

Reducing the dimensionality of your training set before training a model will usually speed up training, but it may not always lead to a better or simpler solution it all depends on the dataset.

Chapter 8: PCA, Preserving, the Variance, Principal Components, Projecting Down to d Dimensions

PCA

- Principal Component Analysis (PCA) is by far the most popular dimensionality reduction algorithm.
- First it identifies the hyperplane that lies closest to the data, and then it projects the data onto it, just like this figure on the following slide ([Figure 8.2](#)).
- The dimensionality reduction process includes preserving the variance, finding the principal component, and projecting down to d Dimensions.

Preserving the Variance

- Choosing the right plane is essential before we can project a training set onto a lower-dimensional hyperplane
- Firstly, we can look at this figure on the following slide ([Figure 8.7](#)).
- Then, we can select the axis that preserves the maximum amount, which is likely to lose less information than the other projections
- The rather simple idea behind PCA is that it is the axis that minimizes the mean squared distance between the original dataset and its projection onto that axis

Principal Components

- PCA identifies the axis with the largest amount of variance in the training set, just like the solid line in [Figure 8.7](#).
- It would also find a second axis, third axis, and so on, which are orthogonal to previous axes. The amount of axes are based on the number of dimensions in the dataset.
- The i^{th} axis is called the i^{th} principal component (PC) of the data (noted as vector c_1 and c_2 in [Figure 8.7](#) and the two arrows that lie on the plane in [Figure 8.2](#)).

Principal Components (NOTE)

- For each principal component, PCA finds a zero-centered unit vector pointing toward the PC.
- Suppose the unit vectors lie on the same axis. In that case, the direction of those returned by CPA is unstable, meaning that if we perturb the training set slightly, the unit vectors may point in the opposite direction as the original ones.
- In other cases, a pair of unit vectors may event rotate or swap (if the variances along these two axes are close).
- However, they will generally still lie on the same axes.

Principal Components (Singular Value Decomposition [SVD])

- Finding principal components (PC) is possible using a standard matrix factorization technique known as Singular Value Decomposition (SVD).
- This technique can decompose the training set matrix X into
- the matrix multiplication of three matrices $U \Sigma V$.
- Look at the following equation :

$$\mathbf{V} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & \cdots & | \end{pmatrix}$$

Note : V contains the unit vectors that define all the principal components

Principal Components (Singular Value Decomposition [SVD])

With NumPy's library, we can use the `svd()` function to obtain all the principal components of the training set, and then extract the two-unit vectors that define the first two PCs:

```
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt.T[:, 0]
c2 = Vt.T[:, 1]
```

Note: Don't forget to center the data first, since PCA assumes that the dataset is centered around the origin

Projecting Down to d Dimensions (1)

- With principal components, we can reduce the dimensionality of the dataset down to d dimensions by projecting it onto the hyperplane defined by the first d principal components
- The hyperplane ensures that the projection will preserve as much variance as possible. On [Figure 8.2](#), 3D dataset is projected down to 2D plane by the first two principal components. This makes the 2D projection looks like the original 3D dataset
- Look at this equation to project the training set down to d dimensions

Projecting Down to d Dimensions (2)

Equation :

$$\mathbf{X}_{d\text{-proj}} = \mathbf{X} \mathbf{W}_d$$

- We can compute the matrix multiplication of the training set matrix $\mathbf{X}$ by the matrix $\mathbf{W}_d$.
- We perform this equation to project the training set onto the hyperplane and obtain a reduced dataset $\mathbf{X}_{d\text{-proj}}$ of dimensionality d .

Projecting Down to d Dimensions (3) Using Python code

- This code projects the training set onto the plane defined by the first two principal components:

```
W2 = Vt.T[:, :2]  
X2D = X_centered.dot(W2)
```

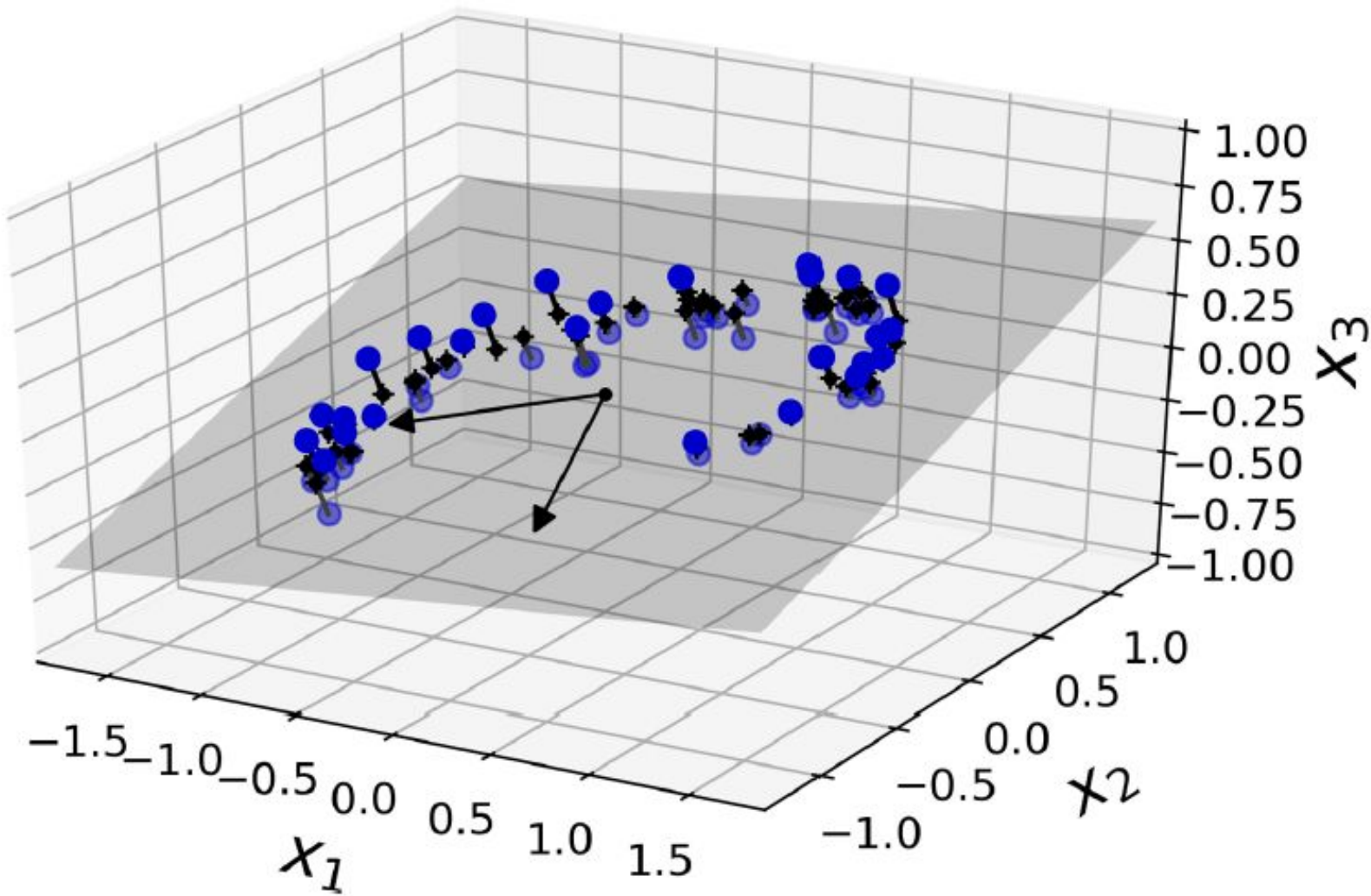


Figure 8-2. A 3D dataset lying close to a 2D subspace

[Back to PCA slide](#)

[Back to Principal Components slide](#)

[Back to Projecting Down to \$d\$ Dimension \(1\) slide](#)

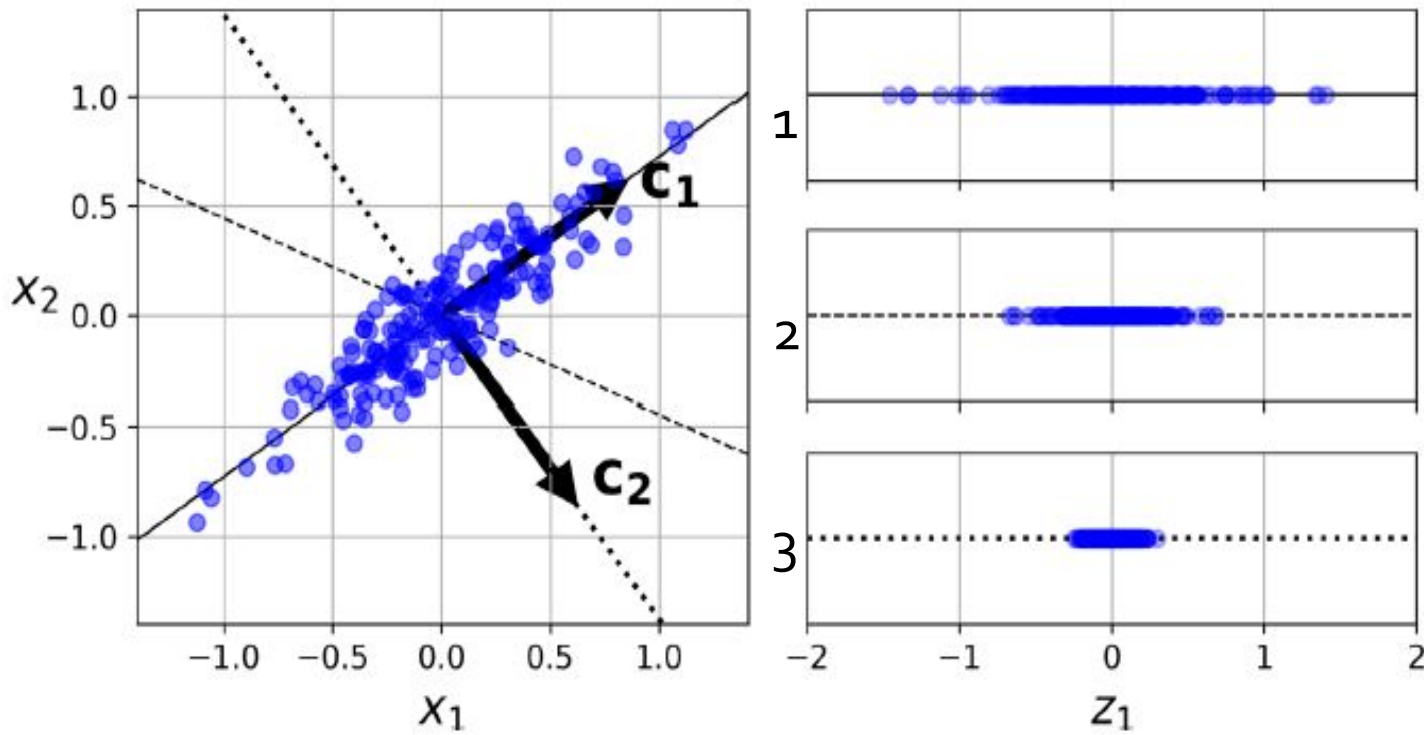


Figure 8-7. Selecting the subspace to project on

There are three (3) projections :

- the solid line preserves the maximum variance (1)
- the projection onto the dashed line preserves an intermediate amount of variance. (2)
- the dotted line preserves very little variance. (3)

Left side : 2D dataset

Right side : result of the projection of the dataset onto each of the axes

[Back to Preserving the Variance slide](#)

[Back to Principal Components slide](#)

Chapter 8: Explained Variance Ratio, Choosing the Right Number of Dimensions

Explained Variance Ratio

The explained variance ratio is the percentage of variance that is attributed by each of the selected components. Ideally, you would choose the number of components to include in your model by adding the explained variance ratio of each component until you reach a total of around 0.8 or 80% to avoid overfitting.

First, let's load the iris dataset for our code-a-long example. The iris dataset is famous and contains measurements for 150 iris flowers from three different species.

```
from sklearn import datasets

import pandas as pd

iris = datasets.load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['Target'] = iris.get('target')

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Choosing the Right Number of Dimensions

- Instead of arbitrarily choosing the number of dimensions to reduce down to, it is simpler to choose the number of dimensions that add up to a sufficiently large portion of the variance (e.g., 95%).
- Unless, of course, you are reducing dimensionality for data visualization—in that case, you will want to reduce the dimensionality down to 2 or 3.

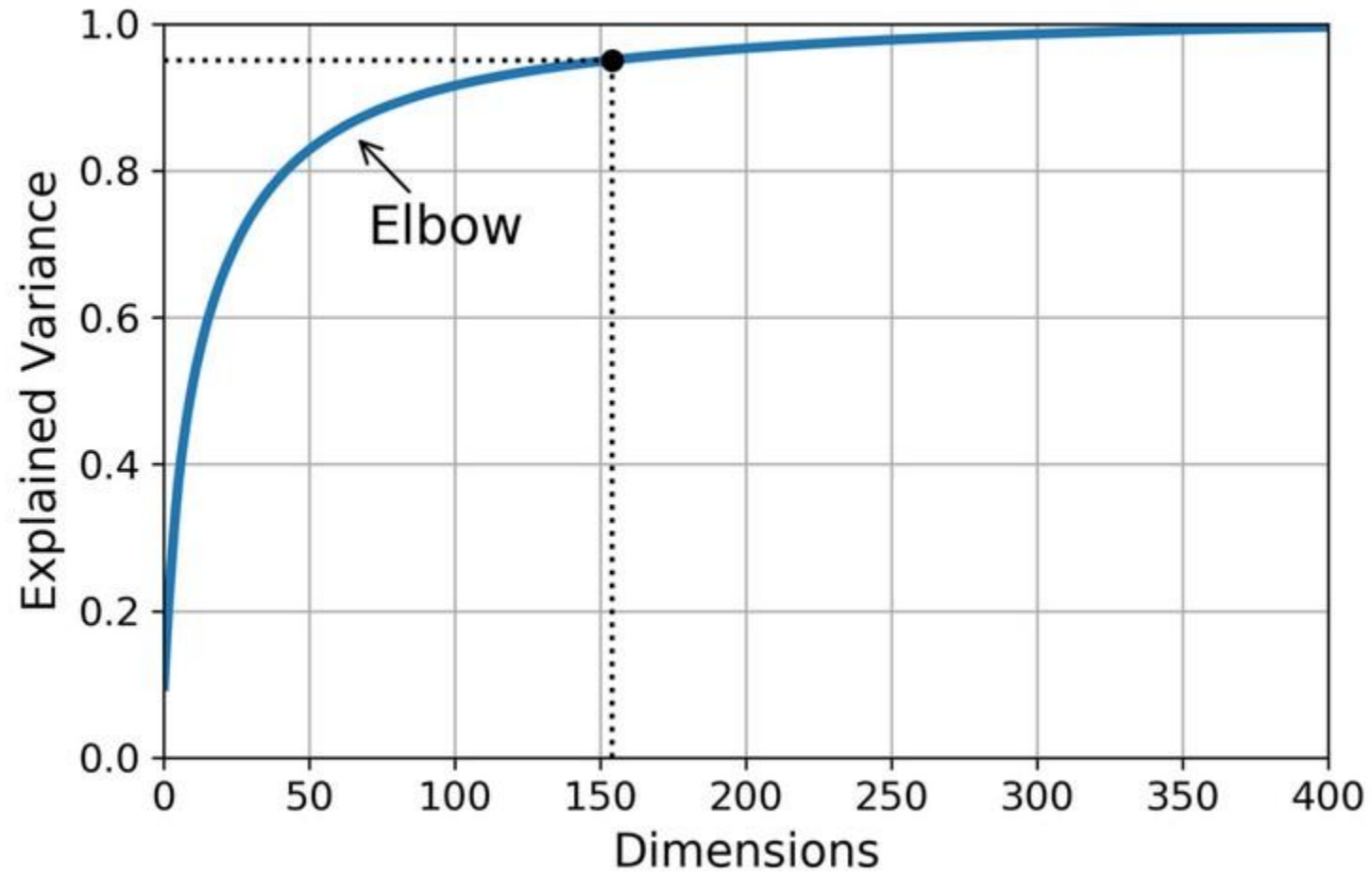


Figure 8-8. Explained variance as a function of the number of dimensions

The following code performs PCA without reducing dimensionality, then computes the minimum number of dimensions required to preserve 95% of the training set's variance:

```
pca = PCA()

pca.fit(X_train)

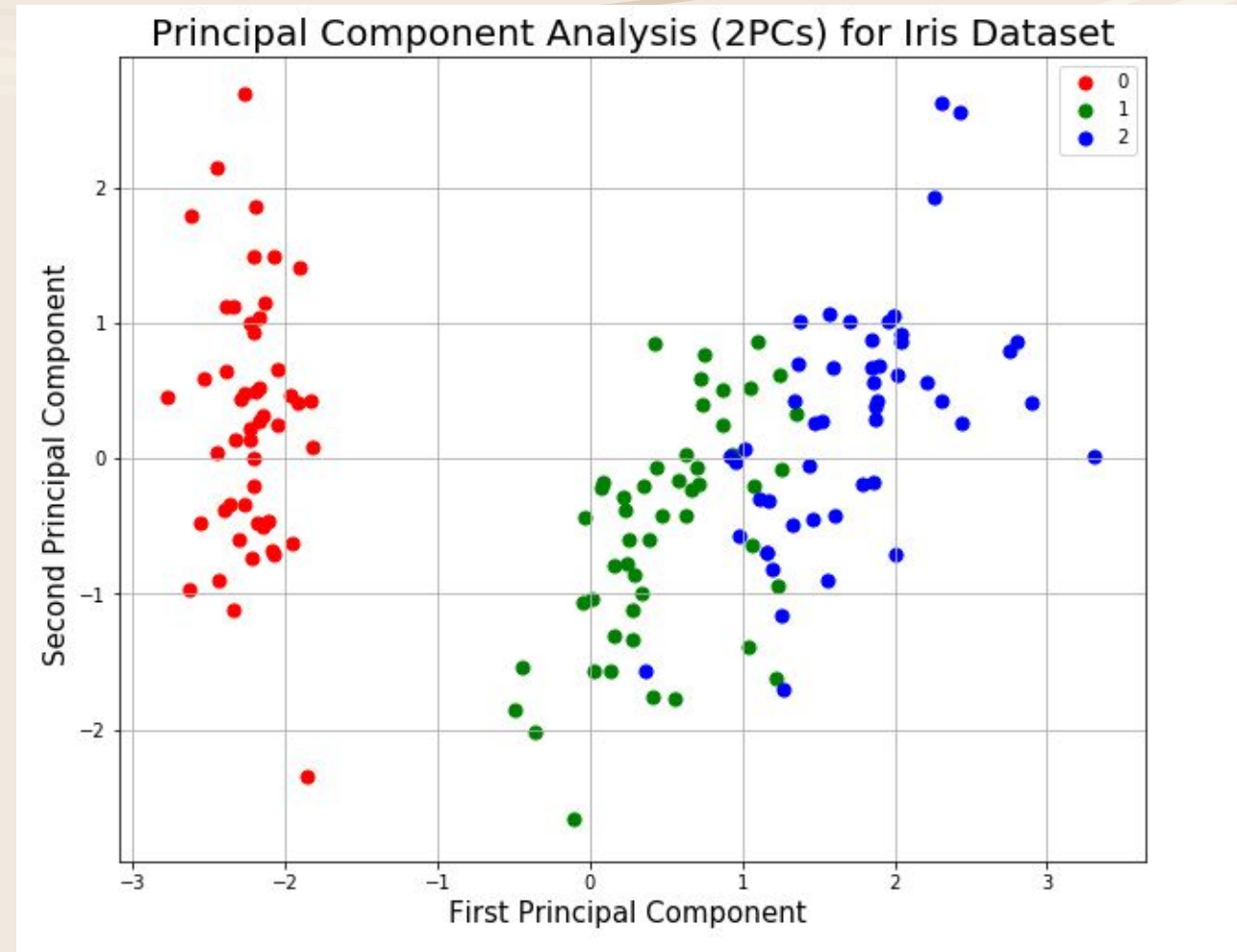
cumsum = np.cumsum(pca.explained_variance_ratio_)

d = np.argmax(cumsum >= 0.95) + 1
```


You could then set `n_components = d` and run PCA again.

But there is a much better option: instead of specifying the number of principal components you want to preserve, you can set `n_components` to be a float between 0.0 and 1.0, indicating the ratio of variance you wish to preserve:

```
pca = PCA(n_components=0.95)  
X_reduced = pca.fit_transform(X_train)
```



Chapter 8: PCA for Compression, Randomized PCA, Incremental PCA

NRP:

Nama:

Kelas:

Kernel PCA

- In Chapter 5 we discussed the kernel trick, a mathematical technique that implicitly maps instances into a very high-dimensional space (called the feature space).
- It turns out that the same trick can be applied to PCA, making it possible to perform complex nonlinear projections for dimensionality reduction. This is called Kernel PCA (kPCA).

Kernel PCA

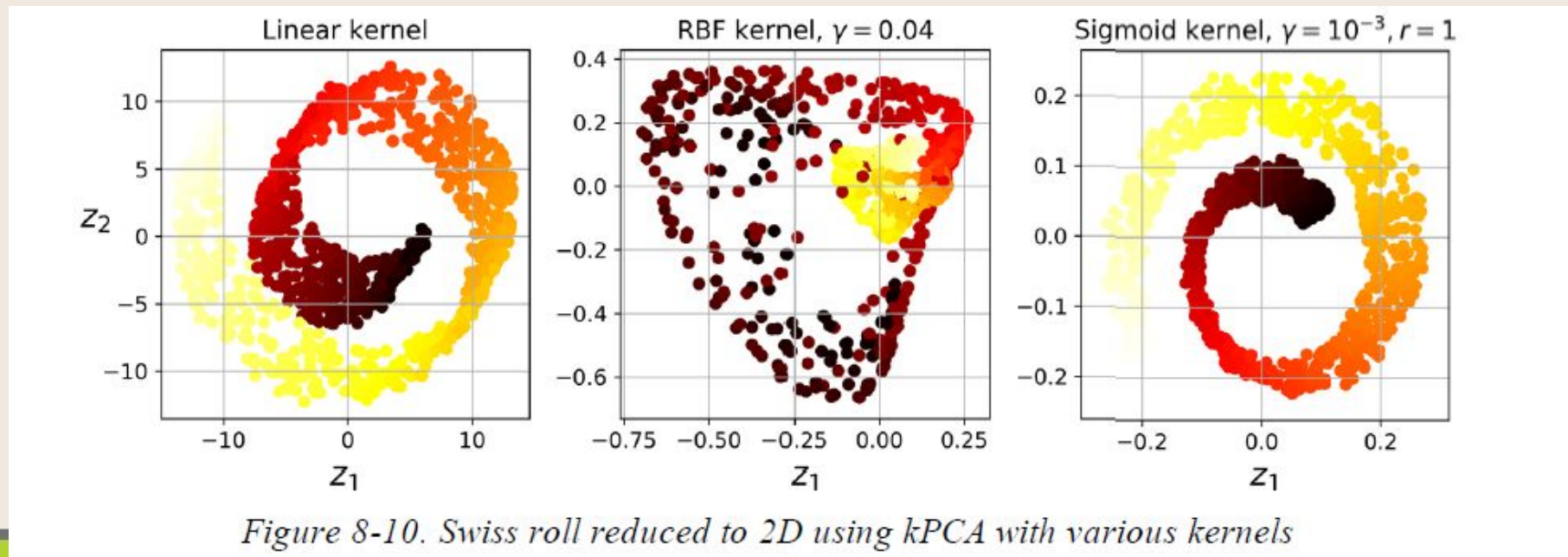
The following code uses Scikit-Learn's kPCA class to perform KernelPCA with an RBF kernel

```
from sklearn.decomposition import KernelPCA

rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.04)
X_reduced = rbf_pca.fit_transform(X)
```

Kernel PCA

Swiss roll, reduced to two dimensions using a linear kernel (equivalent to simply using the PCA class), an RBF kernel, and a sigmoid kernel.



Selecting a Kernel and Tuning Hyperparameters

- As kPCA is an unsupervised learning algorithm, there is no obvious performance measure to help you select the best kernel and hyperparameter values.
- You can use grid search to select the kernel and hyperparameters that lead to the best performance on that task.
- First reducing dimensionality to two dimensions using kPCA, then applying Logistic Regression for classification.
- uses GridSearchCV to find the best kernel and gamma value

Selecting a Kernel and Tuning Hyperparameters

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("kpca", KernelPCA(n_components=2)),
    ("log_reg", LogisticRegression())
])

param_grid = [{
    "kpca__gamma": np.linspace(0.03, 0.05, 10),
    "kpca__kernel": ["rbf", "sigmoid"]
}]

grid_search = GridSearchCV(clf, param_grid, cv=3)
grid_search.fit(X, y)
```

Selecting a Kernel and Tuning Hyperparameters

The best kernel and hyperparameters are then available through the `best_params_` variable:

```
>>> print(grid_search.best_params_)  
{'kpca__gamma': 0.043333333333333335, 'kpca__kernel': 'rbf'}
```

Selecting a Kernel and Tuning Hyperparameters

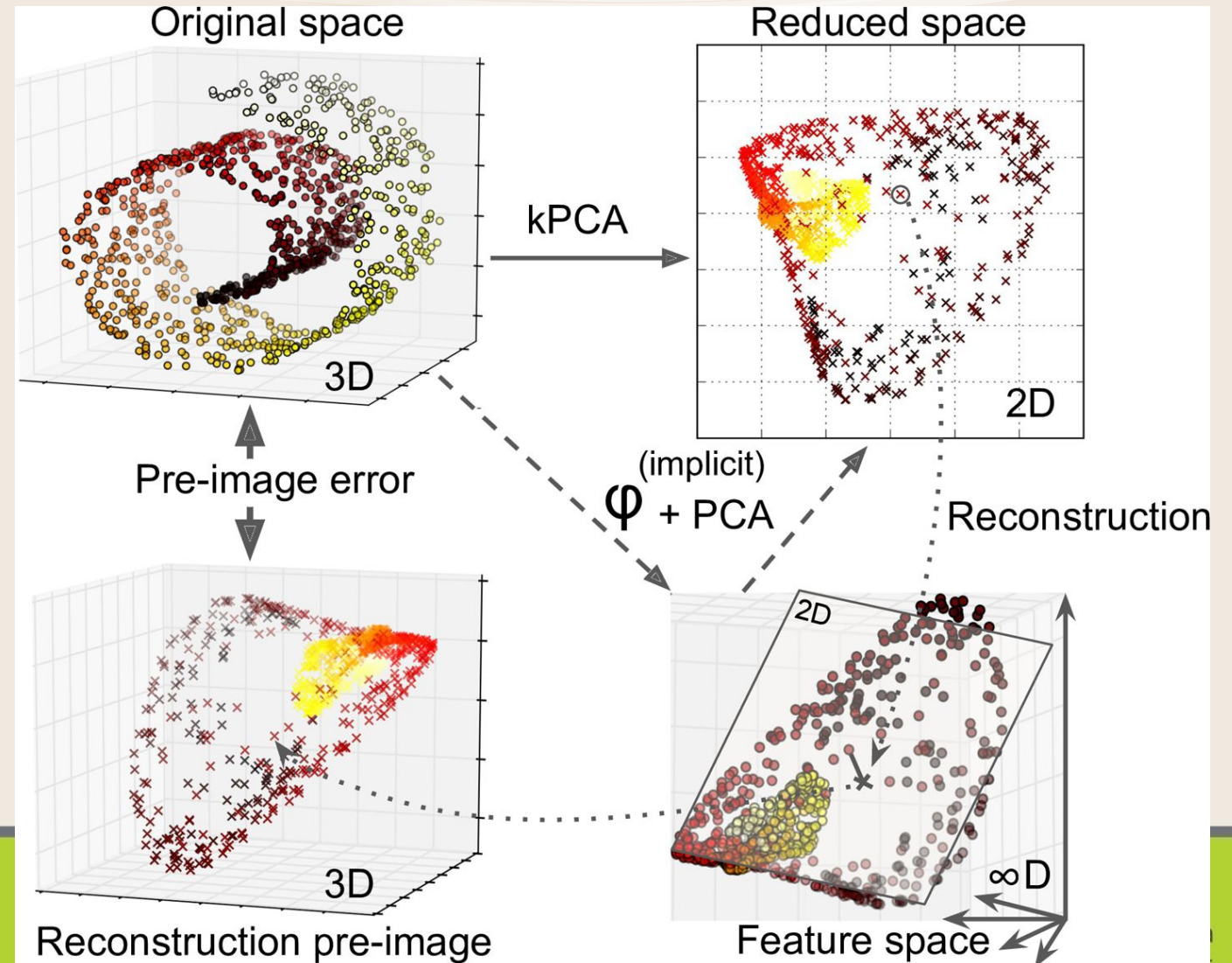
- Another approach, this time entirely unsupervised, is to select the kernel and hyperparameters that yield the lowest reconstruction error.
- this transformation is mathematically equivalent to using the feature map f to map the training set to an infinite-dimensional feature space (bottom right), then projecting the transformed training set down to 2D using linear PCA.

Selecting a Kernel and Tuning Hyperparameters

- Notice that if we could invert the linear PCA step for a given instance in the reduced space, the reconstructed point would lie in feature space, not in the original space
- Since the feature space is infinite-dimensional, we cannot compute the reconstructed point
- Fortunately, it is possible to find a point in the original space that would map close to the reconstructed point.
- This is called the reconstruction pre-image.

Selecting a Kernel and Tuning Hyperparameters

- Once you have this pre-image, you can measure its squared distance to the original instance.
- You can then select the kernel and hyperparameters that minimize this reconstruction pre-image error.



Selecting a Kernel and Tuning Hyperparameters

How to perform this reconstruction :

- One solution is to train a supervised regression model, with the projected instances as the training set and the original instances as the targets.
- Scikit-Learn will do this automatically if you set `fit_inverse_transform=True`. (By default, False and KernelPCA has no `inverse_transform()` method)

```
rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.0433,  
                    fit_inverse_transform=True)  
X_reduced = rbf_pca.fit_transform(X)  
X_preimage = rbf_pca.inverse_transform(X_reduced)
```


Selecting a Kernel and Tuning Hyperparameters

You can then compute the reconstruction pre-image error:

```
>>> from sklearn.metrics import mean_squared_error  
>>> mean_squared_error(X, X_preimage)  
32.786308795766132
```

LLE

- Locally Linear Embedding (LLE) is another powerful nonlinear dimensionality reduction (NLDR) technique.
- Manifold Learning technique that does not rely on projections
- LLE works by first measuring how each training instance linearly relates to its closest neighbors (c.n.),
- looking for a low-dimensional representation of the training set where these local relationships are best preserved (more details shortly).

LLE

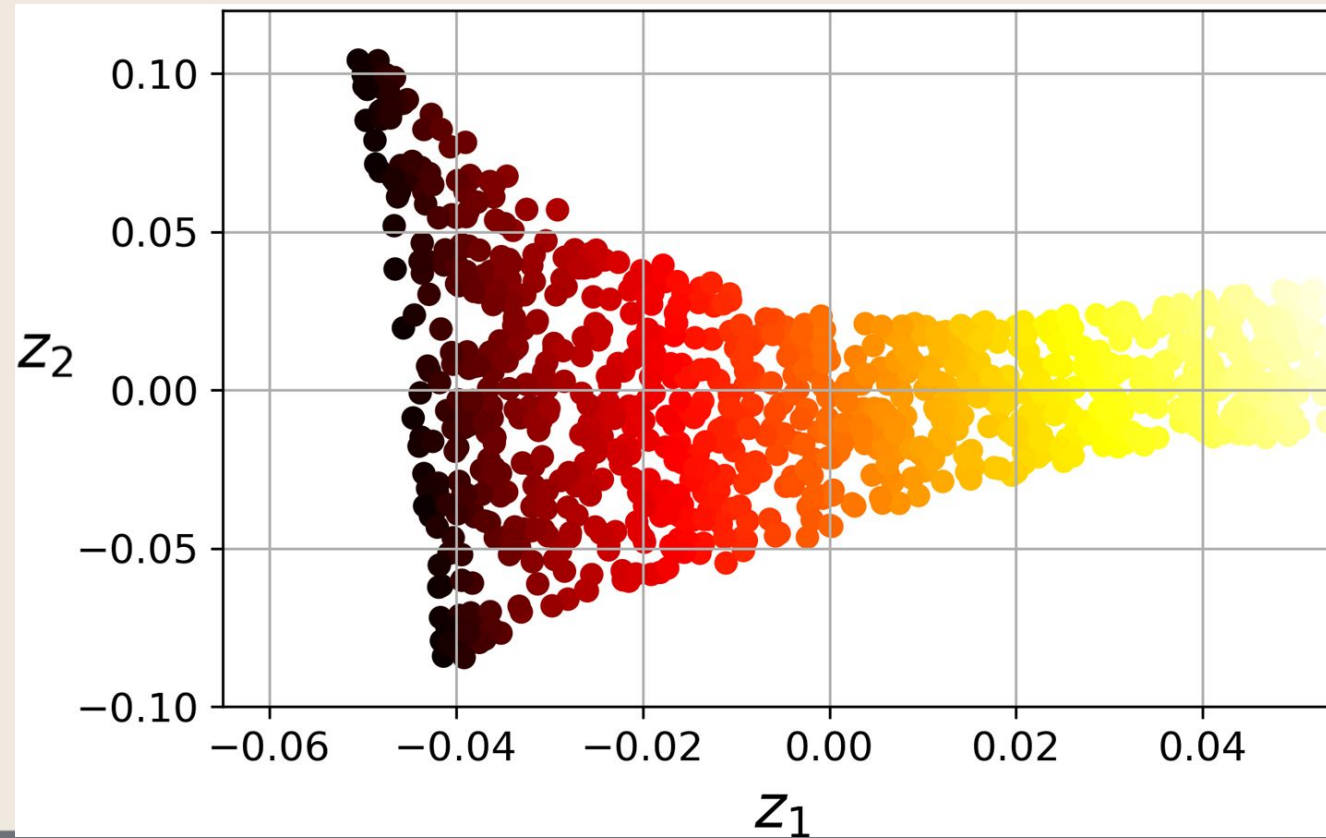
The following code uses Scikit-Learn's LocallyLinearEmbedding class to unroll the Swiss roll:

```
from sklearn.manifold import LocallyLinearEmbedding

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)
X_reduced = lle.fit_transform(X)
```

LLE

- The resulting 2D dataset is shown in Figure 8-12.



LLE

- How LLE works:
 - for each training instance $\mathbf{x}$, the algorithm identifies its k closest neighbors
 - then tries to reconstruct $\mathbf{x}$ as a linear function of these neighbors.

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2$$

subject to
$$\begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases}$$

LLE

- Scikit-Learn's LLE implementation has the following computational complexity:
 - $O(m \log(m)n \log(k))$ for finding the k nearest neighbors,
 - $O(mnk)$ for optimizing the weights, and $O(dm)$ for constructing the lowdimensional representations.
 - Unfortunately, the m in the last term makes this algorithm scale poorly to very large datasets.

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

Other Dimensionality Reduction Techniques

There are many other dimensionality reduction techniques, several of which are available in Scikit-Learn.

- Random Projections
- Multidimensional Scaling (MDS)
- Isomap
- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Linear Discriminant Analysis (LDA)

Other Dimensionality Reduction Techniques

Random Projections

- Projects the data to a lower-dimensional space using a random linear projection
- it turns out that such a random projection is actually very likely to preserve distances well, as was demonstrated mathematically by William B.
- Check out the documentation for the `sklearn.random_projection` package for more details.

Other Dimensionality Reduction Techniques

Multidimensional Scaling (MDS)

- Reduces dimensionality while trying to preserve the distances between the instances.

Isomap

- Creates a graph by connecting each instance to its nearest neighbors, then reduces dimensionality while trying to preserve the geodesic distances between the instances.

Other Dimensionality Reduction Techniques

t-Distributed Stochastic Neighbor Embedding (t-SNE)

- Reduces dimensionality while trying to keep similar instances close and dissimilar instances apart.
- Mostly used for visualization (e.g., to visualize the MNIST images in 2D).

Linear Discriminant Analysis (LDA)

- Classification algorithm
- Good technique to reduce dimensionality before running another classification algorithm such as an SVM classifier.



UNIVERSITAS
KRISTEN
MARANATHA